SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 – PART I

ALGORITHMIC DESIGN THROUGH PSEUDOCODE

Course: Design and Analysis of Algorithms
Course Outcome: CO4 – Solve optimization problems using Dynamic Programming and Greedy strategies
Nature of Assignment: Individual
Duration: One Week
Total Marks: 50
Submission Date: 10-11-26_____

NAME:POOJITHA(192511230)

PURPOSE OF THE ASSIGNMENT

This assignment evaluates your ability to **design an algorithm before implementing it**.

You are expected to demonstrate the following progression:

> **Interpret the Problem → Identify the Decision/State → Design the Logic → Write Pseudocode → Trace → Challenge → Refine**

This is **not a programming assignment**. A compiler or programming language is not required.

All pseudocode must follow the **SIMATS Engineering – HANDBOOK for Algorithmic Pseudocode** provided to you.

TASK 1 – ALGORITHM DESIGN REVIEW

Diagnose and Repair Poor Pseudocode

6 Marks

A logistics company wants to load packages into a delivery vehicle. Each package has a **weight** and **value**. Packages may be divided if necessary. The objective is to maximise the total value loaded without exceeding the vehicle capacity.

A junior engineer proposes:

```
ALGORITHM LoadVehicle

BEGIN
   Read packages
   Arrange packages
   Select the best package

   WHILE vehicle has space DO
      Add suitable package
   END WHILE

   Calculate value
   RETURN result
END
```

Task 1.1 – Diagnose the Design | 2 Marks

Identify **at least five algorithmic deficiencies** in the proposed pseudocode.

Do not focus merely on formatting. Examine aspects such as:

- Inputs and outputs
- Selection criterion
- Meaning of “best”
- Feasibility condition
- Handling of a package that does not completely fit
- Remaining capacity
- Termination condition
- Result construction

For each deficiency, explain briefly why another engineer could not reliably implement the algorithm as written.

Task 1.2 – Repair the Design | 3 Marks

Rewrite the algorithm as **complete, precise and language-independent pseudocode**.

Your design must clearly expose the **greedy-choice rule**.

Task 1.3 – Design Defence | 1 Mark

In not more than **100 words**, explain why your rewritten solution qualifies as pseudocode rather than programming code or informal English.

Evidence Expected

A reasoned diagnosis, corrected pseudocode and concise justification of the design.

TASK 2 – FROM BUSINESS REQUIREMENT TO GREEDY PSEUDOCODE

Job Sequencing with Deadlines

8 Marks

A cloud-service company receives independent computational jobs. Each job requires **one processing slot** and has:

Job ID | Deadline | Profit

A job earns its profit only when completed on or before its deadline. Only one job can be processed in each slot.

The objective is to:

> **Maximise the total profit earned from jobs completed within their deadlines.**

You are **not provided with the algorithm**.

Task 2.1 – Model the Requirement | 2 Marks

Before writing pseudocode, identify:

- Input
- Output
- Objective
- Constraints
- Decision to be made
- Information required to determine whether a job can be scheduled

Task 2.2 – Formulate a Greedy Hypothesis | 1 Mark

State in **one precise sentence** the greedy-choice rule you propose.

Briefly justify why this local decision appears capable of contributing to a globally good solution.

Task 2.3 – Design the Algorithm | 3 Marks

Write a complete pseudocode for your proposed strategy.

Your pseudocode must clearly expose:

- Ordering/selection criterion
- Slot availability
- Feasibility check
- Job placement
- Profit update
- Final schedule
- Total profit

Task 2.4 – Desk-Check | 1 Mark

Manually trace your algorithm using:

Job	Deadline	Profit
A	2	80
B	1	35
C	2	60
D	3	40
E	1	25

Show the **important decisions and state changes**, not merely the final answer.

Task 2.5 – Challenge Your Design | 1 Mark

Change **one input value** in the given instance.

Before tracing the algorithm again:

1. Predict whether the schedule will change.
2. Identify the decision you expect to be affected.
3. Trace only the relevant decisions and determine whether your prediction was correct.

Evidence Expected

Problem model, greedy rule, complete pseudocode, decision trace and modified-case reasoning.

TASK 3 – FROM RECURRENCE TO EXECUTABLE LOGIC

Dynamic Programming – 0/1 Knapsack

9 Marks

A cargo-planning system must select items for a vehicle with limited carrying capacity.

Each item has:

Item ID | Weight | Value

Items are indivisible. An item must either be **completely selected or completely rejected**.

The objective is to maximise total value without exceeding the vehicle capacity.

Task 3.1 – Define the State | 1 Mark

Define precisely:

$$DP[i][w]$$

Your definition must explain exactly what information the state represents.

Task 3.2 – Develop the Recurrence | 2 Marks

Develop the recurrence considering:

- When the current item can be included
- When it cannot be included
- Include versus exclude decision
- Boundary conditions

Explain the **meaning of each term** rather than merely writing the final recurrence.

Task 3.3 – Translate the Recurrence into Pseudocode | 4 Marks

Write complete pseudocode that:

- Initialises the DP structure
- Computes states in a valid dependency order
- Performs the include/exclude decision
- Determines the optimum value
- **Reconstructs the actual items selected**

Returning only the optimum numerical value is insufficient.

Task 3.4 – Desk-Check | 1 Mark

Use the following instance:

Item	Weight	Value
P	2	12
Q	3	20
R	4	25
S	5	30

Vehicle Capacity = 7

Trace the important DP states that lead to the final solution.

You are not required to reproduce unnecessary states if they do not contribute to your explanation.

Task 3.5 – Explain the Dependency | 1 Mark

Explain:

> **Why must certain DP states be available before later states can be computed?**

Relate your explanation to your pseudocode processing order.

Evidence Expected

State definition, recurrence with reasoning, complete pseudocode, solution-reconstruction logic, manual trace and dependency explanation.

TASK 4 – PSEUDOCODE FORENSICS

Find, Explain and Repair an Algorithmic Defect

7 Marks

The following pseudocode is intended to determine whether a string can be completely segmented into words contained in a dictionary.

```
ALGORITHM WordBreak(Text, Dictionary)

INPUT:
   Text – string of length n
   Dictionary – collection of valid words

OUTPUT:
   TRUE if Text can be completely segmented
   FALSE otherwise

BEGIN
   DP[0] ← TRUE

   FOR i ← 1 TO n DO
      DP[i] ← FALSE

      FOR j ← 0 TO i - 1 DO

         IF substring(Text, j, i) is in Dictionary THEN
            DP[i] ← TRUE
            BREAK
         END IF

      END FOR
   END FOR

   RETURN DP[n]
END
```

The pseudocode contains a logical defect.

Task 4.1 – Locate the Defect | 2 Marks

Identify the **exact decision** that is incomplete.

Explain why finding a substring in the dictionary is not, by itself, sufficient to establish that the prefix ending at that position can be segmented.

Task 4.2 – Construct a Failing Input | 2 Marks

Design your own:

- `Text`
- `Dictionary`

for which the faulty algorithm can produce an incorrect result.

Trace the relevant states and demonstrate **where the first incorrect state occurs**.

Task 4.3 – Repair the Algorithm | 2 Marks

Correct **only the required logic**.

Do not replace the entire algorithm with a new solution.

Task 4.4 – Regression Check | 1 Mark

Provide one additional input for which the original algorithm already produced the correct result.

Demonstrate that your correction preserves the correct behaviour.

Evidence Expected

Exact defect, student-designed failing case, state trace, minimal correction and regression evidence.

TASK 5 – SAME GRAPH, DIFFERENT ALGORITHMIC REASONING

Dijkstra, Bellman-Ford and Floyd

7 Marks

Consider the following directed weighted graph:

Edge	Weight
A → B	4
A → C	7
B → C	2
B → D	6
C → D	1
D → E	3

C → 8
E

Task 5.1 – Greedy Shortest-Path Design | 3 Marks

Assume all edge weights are non-negative.

Write pseudocode to determine shortest distances from source **A** using a greedy strategy.

Your pseudocode must explicitly show:

- Initialisation
- Selection of the next vertex
- Relaxation
- Distance update
- Predecessor update
- Termination

Writing **"Apply Dijkstra's algorithm"** is not sufficient.

Task 5.2 – Change the Graph Assumption | 2 Marks

The system must now support graphs containing **negative-weight edges**.

Identify precisely which assumption or decision in your previous greedy reasoning can become unsafe.

Using Bellman-Ford reasoning:

- Write the core pseudocode required for repeated relaxation.
- Explain why a single greedy commitment is no longer sufficient.
- State how a reachable negative-weight cycle can be detected.

Task 5.3 – Change the Requirement | 2 Marks

The organisation now requires shortest paths between **every pair of vertices**.

Define the meaning of:

$$D^{(k)}[i][j]$$

Then write the core state transition for **Floyd's algorithm** in pseudocode.

Explain how the state interpretation differs from the single-source shortest-path approaches.

Evidence Expected

Precise pseudocode, relaxation logic, analysis of the greedy assumption, Bellman-Ford reasoning and Floyd state/transition.

TASK 6 – DESIGN CHALLENGE

Energy-Aware Job Scheduling in a Cloud Data Centre

13 Marks

A cloud data centre operates a specialised **energy-efficient computing server**. During a maintenance window, several computational jobs are waiting to be processed.

Each job has:

- A **start time**
- A **finish time**
- An **energy-saving value** representing the benefit obtained if that job is executed on the energy-efficient server

The server can execute **only one job at a time**. Therefore, jobs whose execution intervals overlap cannot both be selected.

The objective is to:

> **Select a set of non-overlapping jobs that maximises the total energy-saving value.**

You are **not told which algorithm or algorithmic strategy to use**. You must develop the solution from the problem requirements.

Job Data

Job	Start Time	Finish Time	Energy-Saving Value
J1	1	3	20
J2	2	5	50

J3	4	6	30
J4	6	8	40
J5	5	9	60
J6	8	10	30
J7	9	11	50
J8	11	13	40

Assumption: If one job finishes at time t and another starts at time t, the two jobs are considered non-overlapping.

Stage 1 – Interpret the Problem | 2 Marks

From the requirement, clearly identify:

- Input
- Output
- Objective
- Constraints
- Decisions to be made

Do not identify an algorithm merely from the appearance of the problem. Formulate the computational requirement first.

Evidence Expected

A precise abstraction of the scheduling problem.

Stage 2 – Propose the Strategy | 2 Marks

Consider possible ways of constructing the schedule.

You may investigate strategies such as:

- Selecting the job that finishes earliest
- Selecting the job with the highest energy-saving value
- Selecting the shortest-duration job
- Considering combinations of compatible jobs
- Any other strategy you believe is appropriate

Determine which algorithmic strategy you would use.

Justify your choice based on the characteristics of the problem.

Important

A strategy is not justified merely because it produces the correct answer for the given dataset.

Explain **why its decision-making principle is appropriate for the general problem**.

Evidence Expected

Reasoned strategy selection rather than only the name of an algorithm.

Stage 3 – Identify the Critical Information | 2 Marks

Identify what information your algorithm must retain or use when making each decision.

If your design retains information from earlier subproblems:

Clearly define:

- What a state represents
- What decisions lead from one state to another
- What smaller results are required to construct a larger result
- Base condition

If your design makes an immediate selection:

Clearly define:

- The selection criterion
- Why the decision can be made at that stage
- What information is required to determine feasibility
- Why the decision should not need to be reconsidered later

Evidence Expected

A precise state formulation or decision rule consistent with your selected strategy.

Stage 4 – Design Complete Pseudocode | 4 Marks

Write complete, language-independent pseudocode conforming to the **SIMATS Engineering Standard for Algorithmic Pseudocode**.

Your pseudocode must clearly show:

- Input
- Required preprocessing, if any
- Initialisation
- Decision logic
- State updates
- Handling of compatible/incompatible jobs
- Termination
- Reconstruction/reporting of the selected jobs
- Maximum total energy-saving value

Another competent programmer should be able to implement your algorithm directly from the pseudocode without clarification.

Evidence Expected

Complete pseudocode representing the student's own algorithmic design.

Stage 5 – Desk-Check | 1 Mark

Manually execute your algorithm using the **eight-job dataset provided in the problem**.

Show the important:

states → decisions → updates → selected jobs → final value

Report:

- Jobs selected
- Total energy-saving value

Do not provide only the final answer. The decision process must be visible.

Stage 6 – Attack Your Own Design | 2 Mark

Construct a **new set of at least five jobs** specifically intended to challenge your algorithm.

Your test should investigate whether a locally attractive decision can lead to a poor overall result.

Before applying your algorithm:

1. Explain why you believe the constructed input is challenging.
2. Predict how your algorithm will behave.

Then apply your algorithm and determine:

- Whether it still produces the optimal result
- Which decision is critical
- Whether any assumption or decision rule in your design fails

If your algorithm continues to work correctly, explain **why the adversarial input does not defeat it**.

Evidence Expected

A student-designed adversarial dataset, prediction, trace and conclusion.

ASSESSMENT DISTRIBUTION

Task	Competency Assessed	Marks
Task 1	Diagnose and repair algorithmic logic	**6**
Task 2	Design a Greedy algorithm from a requirement	**8**
Task 3	Design DP state, recurrence and pseudocode	**9**
Task 4	Debug using counterexamples and traces	**7**

Task 5	Design and adapt graph optimisation logic	**7**
Task 6	Independently design an algorithm for an unseen problem	**13**
	TOTAL	**50**

SUBMISSION REQUIREMENTS

Your submission must contain the following evidence wherever applicable:

Problem Interpretation → Inputs/Outputs → Objective/Constraints → State or Decision Rule → Pseudocode → Manual Trace → Boundary/Adversarial Test → Interpretation

Important Instructions

1. This is an **individual assignment**.
2. **Do not use a compiler to solve the tasks.** The purpose of this assignment is to demonstrate algorithmic design before implementation.
3. Programming code written in **C, C++, Java, Python or any other programming language will not be accepted as pseudocode**.
4. All pseudocode must follow the **SIMATS Engineering – Handbook for Algorithmic Pseudocode**.
5. Avoid statements such as:
   ```
   Find the best solution
   Process appropriately
   Choose a suitable item
   Apply greedy method
   ```
   Such statements hide the algorithmic decision and will be considered incomplete.
6. Appropriate abstraction such as:
   ```
   SORT Jobs in descending order of profit
   ```
   or
   ```
   EXTRACT_MIN(PriorityQueue)
   ```
   is permitted when the internal implementation of that operation is not being assessed.
7. A correct final answer without the required **algorithmic reasoning, pseudocode or trace** will not receive full credit.
8. The **Day-4 Design Challenge is compulsory**.

TASK 1 – Algorithm Design Review (6 Marks)

Task 1.1 – Diagnose the Design (2 Marks)

#	Deficiency	Why it cannot be implemented reliably
1	**Inputs and outputs are not declared.** "Read packages" does not name the weights, values, count n or the vehicle capacity; "result" is not defined.	An implementer cannot tell what data is supplied, whether capacity is an input, or whether the output is a total value, a list of packages or fractions taken.
2	**Ordering criterion missing.** "Arrange packages" gives no key or direction.	Sorting by weight, by value, by value/weight or by name leads to different (mostly wrong) answers. The greedy-choice rule is hidden.
3	**"Best" is undefined and selected only once.** "Select the best package" is outside the loop and never repeated.	Best could mean highest value, lowest weight or highest value per kg. Selecting once before the loop means no next-best choice is ever made.
4	**Remaining capacity is never initialised, tracked or reduced.** "Vehicle has space" is an informal test.	No variable holds free capacity, so the WHILE condition cannot be evaluated and the vehicle can be overloaded.
5	**No feasibility condition.** "Add suitable package" does not say what makes a package suitable (weight ≤ remaining capacity).	Two engineers will pick different packages. There is also no pointer to the next unused package, so the same package could be added repeatedly.
6	**Partial packages are not handled.** The problem allows division, but nothing says what to do when a package does not fit completely.	Without a rule (take fraction = Remaining / weight), the algorithm either skips the package or overflows. The optimal value is then lost.
7	**Termination is unsafe.** The loop stops only when "vehicle has space" becomes false.	If all packages are used while space is still left, the loop never ends. There is no index bound or exhausted-list test.
8	**Result construction is vague.** "Calculate value" has no accumulator and no record of what was loaded.	The total value is never updated at each addition, and fractions taken are not stored. The load plan cannot be reported.

Task 1.2 – Repair the Design (3 Marks)

Greedy-choice rule: at every step load the package with the highest value-per-unit-weight ratio (V[i] / W[i]) among those not yet considered; take it completely if it fits, otherwise take exactly the fraction that fills the remaining capacity.

```
ALGORITHM FractionalLoadVehicle(W, V, n, Capacity)
INPUT:
   W[1..n]   - weight of each package (W[i] > 0)
   V[1..n]   - value of each package
   n         - number of packages
   Capacity  - vehicle capacity (Capacity >= 0)
OUTPUT:
   X[1..n]     - fraction of package i loaded (0 <= X[i] <= 1)
   TotalValue  - maximum value loaded
BEGIN
   FOR i ← 1 TO n DO
      X[i] ← 0
      Ratio[i] ← V[i] / W[i]
   END FOR
   Order[1..n] ← package indices SORTED in descending order of Ratio
   Remaining ← Capacity
   TotalValue ← 0
   k ← 1
   WHILE k ≤ n AND Remaining > 0 DO
      i ← Order[k]
      IF W[i] ≤ Remaining THEN
         X[i] ← 1
         Remaining ← Remaining - W[i]
         TotalValue ← TotalValue + V[i]
      ELSE
         X[i] ← Remaining / W[i]
         TotalValue ← TotalValue + X[i] * V[i]
         Remaining ← 0
      END IF
      k ← k + 1
   END WHILE
   RETURN X, TotalValue
```

```
END
```

Task 1.3 – Design Defence (1 Mark)

The solution is pseudocode because every input, output and variable is declared and initialised, and every decision is an explicit condition (for example IF W[i] ≤ Remaining). The greedy rule is a precise ordering, descending V/W ratio, and termination is guaranteed by k ≤ n and Remaining > 0. It uses structured keywords (BEGIN/END, FOR, WHILE, IF/ELSE) and the assignment symbol ←. It is not programming code because it has no language syntax, data types or library calls. It is not informal English because no step is vague, such as "best" or "suitable". Any programmer could implement it directly.

TASK 2 – Job Sequencing with Deadlines (8 Marks)

Task 2.1 – Model the Requirement (2 Marks)

Element	Specification
Input	n jobs; each job j has an ID, a deadline d[j] (a positive integer slot number) and a profit p[j].
Output	A schedule (which job occupies which slot) and the total profit of the scheduled jobs.
Objective	Maximise the sum of profits of the jobs completed on or before their deadlines.
Constraints	(1) Each job needs exactly one slot. (2) Each slot holds at most one job. (3) A job in slot t earns its profit only if t ≤ d[j]. (4) Each job is used at most once. (5) Rejected jobs earn nothing.
Decision	For each job: accept it (and choose its slot) or reject it.
Information needed for feasibility	The set of slots already occupied, and the latest free slot t with 1 ≤ t ≤ d[j]. Slots beyond MaxDeadline = max(d) are never useful, so slots 1..MaxDeadline are enough.

Task 2.2 – Greedy Hypothesis (1 Mark)

Greedy rule: consider the jobs in descending order of profit and place each job in the latest free slot on or before its deadline; reject the job if no such slot exists.

Why it can work: processing by profit means a high-profit job is never blocked by a lower-profit one, and a rejected job could only have been added by evicting a job of equal or higher

profit. Choosing the latest free slot keeps the earlier slots free for later jobs with tighter deadlines. Together, no local choice reduces what remains available, which is the exchange argument behind greedy optimality.

Task 2.3 – Design the Algorithm (3 Marks)

```
ALGORITHM JobSequencing(ID, D, P, n)
INPUT:
   ID[1..n]  - job identifiers
   D[1..n]   - deadlines (positive integers)
   P[1..n]   - profits
OUTPUT:
   Slot[1..MaxDeadline] - ID of the job in each slot (EMPTY if unused)
   TotalProfit          - total profit earned
BEGIN
   Order[1..n] ← job indices SORTED in descending order of P
   MaxDeadline ← maximum value in D[1..n]
   FOR t ← 1 TO MaxDeadline DO
      Slot[t] ← EMPTY
   END FOR
   TotalProfit ← 0
   FOR k ← 1 TO n DO
      j ← Order[k]
      t ← D[j]
      WHILE t ≥ 1 AND Slot[t] ≠ EMPTY DO
         t ← t - 1
      END WHILE
      IF t ≥ 1 THEN
         Slot[t] ← ID[j]
         TotalProfit ← TotalProfit + P[j]
      END IF
   END FOR
   OUTPUT Slot[1..MaxDeadline], TotalProfit
END
```

Task 2.4 – Desk-Check (1 Mark)

Sorted order (descending profit): A(80), C(60), D(40), B(35), E(25). MaxDeadline = 3, so Slot

= [EMPTY, EMPTY, EMPTY], TotalProfit = 0.

Job	d	Profit	Slots probed	Decision	Slot[1..3]	TotalProfit
A	2	80	t=2 (free)	Place in slot 2	[–, A, –]	80
C	2	60	t=2 (taken), t=1 (free)	Place in slot 1	[C, A, –]	140
D	3	40	t=3 (free)	Place in slot 3	[C, A, D]	180
B	1	35	t=1 (taken), t=0	Reject	[C, A, D]	180
E	1	25	t=1 (taken), t=0	Reject	[C, A, D]	180

Final schedule: slot 1 = C, slot 2 = A, slot 3 = D. **Total profit = 180.**

Task 2.5 – Challenge Your Design (1 Mark)

Change: profit of B is changed from 35 to 90 (all else unchanged).

1. Prediction: the schedule will change. B now has the highest profit, so it is considered first and takes slot 1, its only legal slot.

2. Decision expected to be affected: the placement of C. C (deadline 2) needs slot 2 or 1. A will take slot 2 and B already holds slot 1, so C should be rejected. D should be unaffected.

3. Trace of relevant decisions:

Job	Slots probed	Decision	Slot[1..3]	Total
B (90)	t=1 (free)	Place in slot 1	[B, –, –]	90
A (80)	t=2 (free)	Place in slot 2	[B, A, –]	170
C (60)	t=2, t=1 (both taken), t=0	Reject	[B, A, –]	170
D (40)	t=3 (free)	Place in slot 3	[B, A, D]	210
E (25)	t=1 (taken), t=0	Reject	[B, A, D]	210

Conclusion: the new schedule is B, A, D with total profit 210. C was displaced by B, exactly as predicted, and the prediction was correct.

TASK 3 – 0/1 Knapsack by Dynamic Programming (9 Marks)

Task 3.1 – Define the State (1 Mark)

DP[i][w] is the maximum total value that can be obtained by choosing a subset of only the first i items (items 1..i), where each item is used at most once and the total weight of the chosen

subset is at most w. Here $0 \le i \le n$ and $0 \le w \le$ Capacity. DP[n][Capacity] is the answer to the original problem.

Task 3.2 – Develop the Recurrence (2 Marks)

- **Boundary conditions:** DP[0][w] = 0 for every w (no items, no value) and DP[i][0] = 0 for every i (no capacity, nothing fits, since all weights are positive).
- **Item i cannot be included** (Wt[i] > w): it does not fit in capacity w, so the best value is the same as with the first i−1 items: DP[i][w] = DP[i−1][w].
- **Item i can be included** (Wt[i] ≤ w): there are two options.
  - Exclude item i: value = DP[i−1][w], the best using the first i−1 items with the same capacity.
  - Include item i: value = Val[i] + DP[i−1][w − Wt[i]]. We gain Val[i], and the remaining capacity w − Wt[i] is filled optimally using the first i−1 items.
- We take the larger of the two, because the optimum must either contain item i or not.

```
DP[0][w] = 0                                          for 0 <= w <= Capacity
DP[i][0] = 0                                          for 0 <= i <= n
DP[i][w] = DP[i-1][w]                                 if Wt[i] > w
DP[i][w] = MAX( DP[i-1][w], Val[i] + DP[i-1][w-Wt[i]] )   if Wt[i] <= w
```

Task 3.3 – Pseudocode (4 Marks)

```
ALGORITHM Knapsack01(Wt, Val, n, Capacity)
INPUT:
   Wt[1..n]  - positive integer weights
   Val[1..n] - values
   n         - number of items
   Capacity  - vehicle capacity (integer >= 0)
OUTPUT:
   BestValue - maximum total value
   Chosen    - list of items selected
BEGIN
   // Initialisation (boundary conditions)
   FOR w ← 0 TO Capacity DO
      DP[0][w] ← 0
   END FOR
   FOR i ← 0 TO n DO
```

```
    DP[i][0] ← 0
  END FOR

  // Fill table row by row: row i uses only row i-1
  FOR i ← 1 TO n DO
    FOR w ← 1 TO Capacity DO
      IF Wt[i] > w THEN
        DP[i][w] ← DP[i-1][w]
      ELSE
        Exclude ← DP[i-1][w]
        Include ← Val[i] + DP[i-1][w - Wt[i]]
        IF Include > Exclude THEN
          DP[i][w] ← Include
        ELSE
          DP[i][w] ← Exclude
        END IF
      END IF
    END FOR
  END FOR

  BestValue ← DP[n][Capacity]

  // Reconstruction of the selected items
  Chosen ← empty list
  w ← Capacity
  FOR i ← n DOWNTO 1 DO
    IF DP[i][w] ≠ DP[i-1][w] THEN
      ADD item i to Chosen
      w ← w - Wt[i]
    END IF
  END FOR
  RETURN BestValue, Chosen
END
```

Reconstruction logic: if DP[i][w] differs from DP[i−1][w], the value could only have been achieved by including item i, so i is selected and the capacity is reduced by Wt[i]. If the two are

equal, item i is not needed and we move to row i−1 with the same w.

Task 3.4 – Desk-Check (1 Mark)

Items: P(2, 12), Q(3, 20), R(4, 25), S(5, 30); Capacity = 7. Completed table (row 0 and column 0 are all 0):

DP[i][w]	w=0	w=1	w=2	w=3	w=4	w=5	w=6	w=7
i=0 (none)	0	0	0	0	0	0	0	0
i=1 (P)	0	0	12	12	12	12	12	12
i=2 (Q)	0	0	12	20	20	32	32	32
i=3 (R)	0	0	12	20	25	32	37	45
i=4 (S)	0	0	12	20	25	32	37	45

Key states that lead to the solution:

- DP[2][5] = max(DP[1][5] = 12, 20 + DP[1][2] = 32) = 32 (P and Q together).
- DP[3][7] = max(DP[2][7] = 32, 25 + DP[2][3] = 45) = **45**. R is included, leaving capacity 3, which Q fills with value 20.
- DP[4][7] = max(DP[3][7] = 45, 30 + DP[3][2] = 42) = 45. S does not improve the result.

Reconstruction: i=4, w=7: DP[4][7] = DP[3][7], so S is not selected. i=3, w=7: DP[3][7] = 45 ≠ DP[2][7] = 32, so **R is selected** and w = 3. i=2, w=3: DP[2][3] = 20 ≠ DP[1][3] = 12, so **Q is selected** and w = 0. i=1, w=0: DP[1][0] = DP[0][0], so P is not selected.

Result: items {Q, R}, total weight 3 + 4 = 7 ≤ 7, **maximum value = 45**.

Task 3.5 – Explain the Dependency (1 Mark)

Every state DP[i][w] is built from two states of the previous row: DP[i−1][w] (exclude) and DP[i−1][w − Wt[i]] (include). These smaller subproblems must already hold their optimal values, otherwise the max would be taken over undefined or incorrect numbers. This is why the pseudocode first fills row 0 and column 0 (the base cases), and then runs the outer loop with i increasing from 1 to n, so that every cell of row i−1 exists before row i is computed. Within a row, the order of w does not matter because no cell depends on another cell of the same row. In the reconstruction, the loop runs in the opposite direction (i from n down to 1) because each choice at row i tells us which cell of row i−1 to examine next. The full table must therefore be kept.

TASK 4 – Pseudocode Forensics: WordBreak (7 Marks)

Convention: substring(Text, j, i) means the characters at positions j to i−1 (0-indexed, i

exclusive), so DP[i] describes the prefix of length i.

Task 4.1 – Locate the Defect (2 Marks)

Defective decision: the condition IF substring(Text, j, i) is in Dictionary THEN is incomplete. It never checks that the prefix before the split point, Text[0..j), is itself segmentable, i.e. that DP[j] = TRUE.

Why a dictionary hit is not enough: DP[i] = TRUE means the entire prefix of length i can be split into dictionary words. If the last word Text[j..i) is a dictionary word but the prefix Text[0..j) cannot be segmented, then some part of the string before the word is unaccounted for. The word at the end does not repair the gap. A valid segmentation needs both parts: DP[j] is TRUE and the final piece is a word.

Task 4.2 – Failing Input (2 Marks)

Text = "catxdog" (n = 7); **Dictionary** = {"cat", "dog"}. The correct answer is FALSE, because the letter x cannot be covered by any word.

i	Substrings tested (j)	Dictionary hits	DP[i] (faulty)	Correct DP[i]
1, 2	c, ca, a …	none	FALSE	FALSE
3	j=0 "cat"	"cat" (j=0, DP[0]=T)	TRUE	TRUE
4	catx, atx, tx, x	none	FALSE	FALSE
5, 6	…d, …do, o	none	FALSE	FALSE
7	j=0..3 no; **j=4 "dog"**	"dog" with **DP[4] = FALSE**	**TRUE (wrong)**	FALSE

First incorrect state: DP[7]. At i = 7, j = 4 the substring "dog" is found, and the faulty algorithm sets DP[7] = TRUE and BREAKs without checking that DP[4] is FALSE. The algorithm returns **TRUE**, which is incorrect.

Task 4.3 – Repair the Algorithm (2 Marks)

Only the IF condition changes; everything else in the algorithm is untouched:

> Original: IF substring(Text, j, i) is in Dictionary THEN
> Corrected: IF DP[j] = TRUE AND substring(Text, j, i) is in Dictionary THEN

DP[j] is always available when i is processed, because j < i and DP is filled in increasing order of i. Re-running the failing input: at i = 7, j = 4, DP[4] = FALSE, so the condition fails. No other j gives a dictionary word, so DP[7] = FALSE and the algorithm correctly returns FALSE.

Task 4.4 – Regression Check (1 Mark)

Input: Text = "catdog", Dictionary = {"cat", "dog"}. The original algorithm returns the correct answer TRUE.

i	Split tested	Original	Corrected (DP[j] AND word)	DP[i]
3	j=0, "cat"	word found	DP[0]=T and "cat" ∈ Dict	TRUE
6	j=3, "dog"	word found	DP[3]=T and "dog" ∈ Dict	TRUE
1, 2, 4, 5	no dictionary word	FALSE	FALSE	FALSE

Both versions give DP[6] = TRUE. The extra check DP[j] = TRUE is satisfied wherever a valid segmentation exists, so correct behaviour is preserved.

TASK 5 – Dijkstra, Bellman-Ford and Floyd (7 Marks)

Task 5.1 – Greedy Shortest-Path Design (3 Marks)

```
ALGORITHM GreedyShortestPath(V, E, w, Source)
INPUT:
   V - set of vertices;  E - set of directed edges (u, v)
   w(u, v) >= 0 - weight of each edge;  Source - start vertex
OUTPUT:
   Dist[v] - shortest distance from Source;  Pred[v] - predecessor on path
BEGIN
   FOR each vertex v in V DO                    // initialisation
      Dist[v] ← INF
      Pred[v] ← NIL
   END FOR
   Dist[Source] ← 0
   Q ← V                                  // vertices not yet finalised
   WHILE Q is not empty DO                    // termination: Q empty
      u ← EXTRACT_MIN(Q) by Dist              // greedy selection
      IF Dist[u] = INF THEN
         BREAK                            // remaining vertices unreachable
      END IF
      remove u from Q
      FOR each edge (u, v) in E with v in Q DO
```

```
        IF Dist[u] + w(u, v) < Dist[v] THEN    // relaxation test
            Dist[v] ← Dist[u] + w(u, v)        // distance update
            Pred[v] ← u                        // predecessor update
        END IF
    END FOR
  END WHILE
  RETURN Dist, Pred
END
```

Trace on the given graph (source A):

Step	**Selected u (Dist)**	**Relaxations**	**Dist [A,B,C,D,E]**
Init	–	–	[0, INF, INF, INF, INF]
1	A (0)	B = 4 (Pred A); C = 7 (Pred A)	[0, 4, 7, INF, INF]
2	B (4)	C: 4+2 = 6 < 7 → 6 (Pred B); D = 10 (Pred B)	[0, 4, 6, 10, INF]
3	C (6)	D: 6+1 = 7 < 10 → 7 (Pred C); E = 14 (Pred C)	[0, 4, 6, 7, 14]
4	D (7)	E: 7+3 = 10 < 14 → 10 (Pred D)	[0, 4, 6, 7, 10]
5	E (10)	no outgoing edges; Q empty, stop	[0, 4, 6, 7, 10]

Shortest distances from A: B = 4, C = 6, D = 7, E = 10. Shortest path to E: A → B → C → D → E.

Task 5.2 – Change the Graph Assumption (2 Marks)

Unsafe assumption: the greedy step "once u is the unfinalised vertex with minimum Dist, Dist[u] is final and never needs to be revisited" is safe only if edge weights are non-negative, because any other route to u passes through vertices at least as far away and cannot get shorter by adding more edges. With a negative edge, a longer-looking route can become shorter later.

Example: edges A→B = 3, A→C = 5, C→B = −4. The greedy algorithm finalises B with distance 3 before C is processed, but the real shortest distance is 5 − 4 = 1 through C.

Bellman-Ford core (repeated relaxation):

```
FOR each vertex v in V DO
```

```
    Dist[v] ← INF;  Pred[v] ← NIL
END FOR
Dist[Source] ← 0
FOR pass ← 1 TO |V| - 1 DO
    Changed ← FALSE
    FOR each edge (u, v) with weight w(u, v) in E DO
        IF Dist[u] ≠ INF AND Dist[u] + w(u, v) < Dist[v] THEN
            Dist[v] ← Dist[u] + w(u, v)
            Pred[v] ← u
            Changed ← TRUE
        END IF
    END FOR
    IF Changed = FALSE THEN BREAK
END FOR
// negative-cycle check
FOR each edge (u, v) with weight w(u, v) in E DO
    IF Dist[u] ≠ INF AND Dist[u] + w(u, v) < Dist[v] THEN
        RETURN "NEGATIVE CYCLE REACHABLE FROM SOURCE"
    END IF
END FOR
RETURN Dist, Pred
```

Why one greedy commitment is not enough: with negative edges, a distance can still decrease after a vertex has been the minimum. After pass k, Bellman-Ford guarantees correct distances for all shortest paths using at most k edges. Every vertex must therefore stay open to improvement until all paths of up to |V|−1 edges have been tried. Nothing can be finalised early.

Detecting a negative cycle: a shortest path without cycles has at most |V|−1 edges, so |V|−1 passes settle all distances. If an extra (|V|-th) pass can still relax any edge, then a path with |V| or more edges is shorter, which is possible only if a negative-weight cycle is reachable from the source.

Task 5.3 – Change the Requirement: All-Pairs (2 Marks)

State meaning: D(k)[i][j] is the length of the shortest path from vertex i to vertex j whose intermediate vertices (all vertices other than i and j) belong only to the set {1, 2, …, k}. D(0)[i][j] is the direct edge weight w(i, j) (0 if i = j, INF if there is no edge).

Transition: D(k)[i][j] = MIN(D(k−1)[i][j], D(k−1)[i][k] + D(k−1)[k][j]). The first term is the best path that does not use vertex k; the second is the best path that goes i → k → j, with both halves using only intermediates from {1..k−1}.

```
FOR i ← 1 TO n DO
   FOR j ← 1 TO n DO
      IF i = j THEN D[i][j] ← 0
      ELSE IF edge (i, j) exists THEN D[i][j] ← w(i, j)
      ELSE D[i][j] ← INF
   END FOR
END FOR
FOR k ← 1 TO n DO
   FOR i ← 1 TO n DO
      FOR j ← 1 TO n DO
         IF D[i][k] ≠ INF AND D[k][j] ≠ INF AND D[i][k] + D[k][j] < D[i][j] THEN
            D[i][j] ← D[i][k] + D[k][j]
         END IF
      END FOR
   END FOR
END FOR
```

How the interpretation differs: in Dijkstra and Bellman-Ford the state is one value per vertex, Dist[v], for a single fixed source. It improves through greedy extraction (Dijkstra) or through the number of edges allowed (Bellman-Ford). In Floyd's algorithm the state is a value for every pair (i, j), and the dimension that grows is the set of vertices allowed as intermediates. It is a dynamic-programming recurrence on that set, not a greedy commitment, and it processes the vertices in the order 1..n. After the last k, D(n)[i][j] holds all-pairs shortest distances.

TASK 6 – Design Challenge: Energy-Aware Job Scheduling (13 Marks)

Stage 1 – Interpret the Problem (2 Marks)

Element	**Specification**
Input	n jobs; job j has start time S[j], finish time F[j] (S[j] < F[j]) and energy-saving value E[j] > 0.
Output	A subset of jobs that can run on the server, and the total energy-saving value of that subset.
Objective	Maximise the sum of E[j] over the selected jobs.

Element	Specification
Constraints	The server runs one job at a time. Two selected jobs a and b must not overlap: F[a] ≤ S[b] or F[b] ≤ S[a]. Touching intervals (finish = start) are allowed.
Decisions	For every job: select it or reject it, subject to compatibility with the other selected jobs.

Stage 2 – Propose the Strategy (2 Marks)

Four candidate strategies were examined on the given data and in general.

Strategy	Result on given data	Why it fails or works in general
Earliest finish time first	J1, J3, J4, J6, J8 = 20+30+40+30+40 = **160**	Optimal only for the unweighted version (maximise the count). It ignores values, so it takes the low-value J1 and J3 and loses J2 + J5 + J7 (50+60+50).
Highest value first	J5, J2, J7, J8 = **200**	Correct here by coincidence. Counter-example: one job [0,10] with value 10 against ten unit jobs [i, i+1] with value 9 each. Greedy gets 10 but the optimum is 90.
Shortest duration first	J1, J3, J4, J6, J8 = **160**	Duration says nothing about value, so it can pick many cheap short jobs and block a valuable long one.
Try all compatible combinations	Correct	Examines 2^n subsets, which is exponential and impractical.

Chosen strategy: Dynamic Programming. No simple local criterion (finish time, value or length) is safe once jobs carry different weights, so the greedy-choice property fails. The problem does, however, have the two properties that justify DP:

- **Optimal substructure:** after sorting by finish time, the optimal schedule for the first j jobs either excludes job j (so it is an optimal schedule for the first j−1 jobs) or includes job j (so the rest of it is an optimal schedule over the jobs compatible with j).

- **Overlapping subproblems:** many jobs share the same prefix subproblems, for example OPT[p(j)] and OPT[j−1] are reused. Computing each prefix once gives a polynomial algorithm.

Because the choice rests on these properties rather than on the dataset, the method is correct for every input, not only for the eight given jobs.

Stage 3 – Identify the Critical Information (2 Marks)

- **Preprocessing:** sort jobs by finish time F ascending, so job indices follow finish order.
- **P[j] (compatibility pointer):** the largest index k < j with F[k] ≤ S[j], or 0 if none. Because of the sorting, jobs 1..P[j] are all compatible with job j (they finish by S[j]), and jobs P[j]+1..j−1 all overlap j (they finish after S[j] but start before F[j]).
- **State:** OPT[j] = maximum total energy saving obtainable using only jobs 1..j (in finish order), with no two overlapping.
- **Decisions leading between states:** Exclude job j gives OPT[j−1]. Include job j gives E[j] + OPT[P[j]].
- **Smaller results needed:** OPT[j−1] and OPT[P[j]], both with index less than j.
- **Base condition:** OPT[0] = 0 (no jobs, no value).

```
OPT[0] = 0
OPT[j] = MAX( OPT[j-1],  E[j] + OPT[P[j]] )      for j = 1..n
```

Stage 4 – Complete Pseudocode (4 Marks)

```
ALGORITHM MaxEnergySchedule(ID, S, F, E, n)
INPUT:
   ID[1..n] - job identifiers      S[1..n] - start times
   F[1..n]  - finish times         E[1..n] - energy-saving values
   (jobs a and b are compatible if F[a] <= S[b] or F[b] <= S[a])
OUTPUT:
   Selected - list of chosen job IDs in time order
   MaxValue - maximum total energy-saving value
BEGIN
   // Preprocessing: order jobs by finish time
   SORT arrays ID, S, F, E together in ascending order of F


   // Compute the compatibility pointer for every job
   FOR j ← 1 TO n DO
```

```
    P[j] ← 0
    k ← j - 1
    WHILE k ≥ 1 DO
        IF F[k] ≤ S[j] THEN
            P[j] ← k
            BREAK
        END IF
        k ← k - 1
    END WHILE
END FOR

// Initialisation
OPT[0] ← 0

// Decision logic and state updates
FOR j ← 1 TO n DO
    Exclude ← OPT[j - 1]                // job j rejected
    Include ← E[j] + OPT[P[j]]          // job j accepted; only compatible
                                        //   jobs 1..P[j] remain usable
    IF Include > Exclude THEN
        OPT[j] ← Include
        Choice[j] ← TAKE
    ELSE
        OPT[j] ← Exclude
        Choice[j] ← SKIP
    END IF
END FOR

// Reconstruction
Selected ← empty list
j ← n
WHILE j ≥ 1 DO
    IF Choice[j] = TAKE THEN
        INSERT ID[j] at the front of Selected
        j ← P[j]                        // jump over incompatible jobs
    ELSE
```

```
        j ← j - 1
      END IF
    END WHILE

    MaxValue ← OPT[n]
    OUTPUT Selected, MaxValue
END
```

Handling of jobs: a job j with P[j] = k is compatible with exactly jobs 1..k (the touching case F[k] = S[j] is compatible because of ≤). Incompatible jobs are skipped by the jump j ← P[j] during reconstruction, and by the use of OPT[P[j]] during computation. Ties (Include = Exclude) choose SKIP, which gives a valid optimal answer either way. For n = 0 the loops do not run and the output is an empty list with value 0.

Stage 5 – Desk-Check (1 Mark)

The jobs are already in finish-time order (3, 5, 6, 8, 9, 10, 11, 13), so indices 1..8 correspond to J1..J8.

j	Job (S–F, E)	P[j]	Exclude = OPT[j−1]	Include = E + OPT[P[j]]	OPT[j]	Choice
1	J1 (1–3, 20)	0	0	20 + 0 = 20	20	TAKE
2	J2 (2–5, 50)	0	20	50 + 0 = 50	50	TAKE
3	J3 (4–6, 30)	1	50	30 + 20 = 50	50	SKIP (tie)
4	J4 (6–8, 40)	3	50	40 + 50 = 90	90	TAKE
5	J5 (5–9, 60)	2	90	60 + 50 = 110	110	TAKE
6	J6 (8–10, 30)	4	110	30 + 90 = 120	120	TAKE
7	J7 (9–11, 50)	5	120	50 + 110 = 160	160	TAKE
8	J8 (11–13, 40)	7	160	40 + 160 = 200	200	TAKE

P[j] checks: J3 starts at 4, J1 finishes at 3 so P = 1 (J2 finishes at 5 > 4). J4 starts at 6 and J3 finishes at 6, so P = 3 (touching is allowed). J5 starts at 5 and J2 finishes at 5, so P = 2. J6 starts at 8 and J4 finishes at 8, so P = 4. J7 starts at 9 and J5 finishes at 9, so P = 5. J8 starts at 11 and J7 finishes at 11, so P = 7.

Reconstruction: j=8 TAKE (J8) → j=P[8]=7; j=7 TAKE (J7) → j=5; j=5 TAKE (J5) → j=2;

j=2 TAKE (J2) → j=0, stop.

Jobs selected: J2 (2–5), J5 (5–9), J7 (9–11), J8 (11–13). Total energy-saving value = 50 + 60 + 50 + 40 = 200. No two selected intervals overlap.

Stage 6 – Attack Your Own Design (2 Marks)

Adversarial job set (six jobs, already listed in finish-time order; the two jobs finishing at 10 are ordered D then A):

Index	Job	Start	Finish	Value
1	F	0	2	1
2	B	0	4	30
3	C	4	7	30
4	E	3	8	25
5	D	7	10	30
6	A	0	10	50

Why it is challenging: A is a single job with the highest value (50), which tempts a value-greedy rule, yet it blocks the whole window 0–10. F finishes earliest, which tempts an earliest-finish rule, but its tiny value of 1 excludes B and later forces C and D. E overlaps both C and D and looks reasonable. The best answer is the chain B, C, D (30+30+30 = 90), which is not the highest-value job and does not start with the earliest-finishing job.

Prediction: value-greedy will fail with A alone = 50. Earliest-finish will choose F, C, D = 61 and fail. My DP should compare Include and Exclude at every job and reach 90, with the critical decisions at j = 3 (take C), j = 4 (skip E) and j = 6 (skip A).

P values: P[1] = 0, P[2] = 0, P[3] = 2 (B finishes at 4 = start of C), P[4] = 1 (F finishes at 2 ≤ 3), P[5] = 3 (C finishes at 7 = start of D), P[6] = 0.

j	Job	P[j]	Exclude	Include	OPT[j]	Choice
1	F	0	0	1 + 0 = 1	1	TAKE
2	B	0	1	30 + 0 = 30	30	TAKE
3	C	2	30	30 + 30 = 60	60	TAKE
4	E	1	60	25 + 1 = 26	60	SKIP
5	D	3	60	30 + 60 = 90	90	TAKE

j	Job	P[j]	Exclude	Include	OPT[j]	Choice
6	A	0	90	50 + 0 = 50	90	SKIP

Reconstruction: j=6 SKIP → 5; j=5 TAKE D → P[5]=3; j=3 TAKE C → P[3]=2; j=2 TAKE B → P[2]=0, stop. **Selected = B, C, D; value = 90.**

Conclusion: the algorithm remains optimal. The result is 90 against 50 (value-greedy) and 61 (earliest-finish greedy). No other compatible set does better: F+C+D = 61, F+E = 26, A alone = 50. The critical decision is at j = 6. Locally A looks very attractive (value 50), but Include = 50 + OPT[0] = 50 is compared with OPT[5] = 90, so the earlier combination is kept. The design does not rely on any single local choice, because every job is evaluated against the best result of its smaller subproblems. The only assumptions are (1) correct sorting by finish time, (2) a correct P[j] using F[k] ≤ S[j], and (3) positive values. All three hold here, so no decision rule fails. A wrong sort order or an incorrect P[j] (for example F[k] < S[j], which would wrongly forbid touching jobs) is what would break the design.

Complexity: as written, $O(n^2)$ for computing P ($O(n \log n)$ if binary search is used for P[j]) and O(n) for the DP and reconstruction, plus $O(n \log n)$ for sorting.